

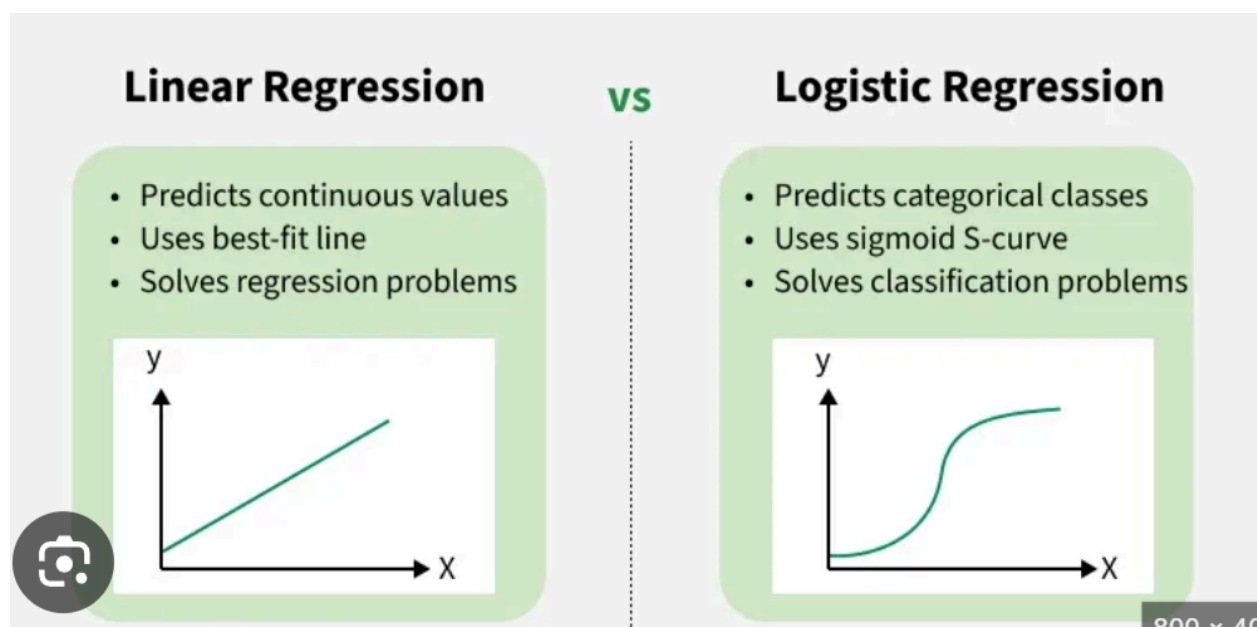
AUTISM PREDICTION EASY EXPLANATION -CONTENT

Logistic Regression

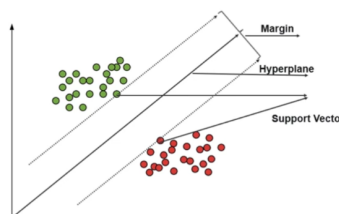
Logistic Regression is a supervised machine learning algorithm used for binary classification problems.

- Probability close to 0 → No ASD
- Probability close to 1 → ASD

If the probability is greater than 0.5, the model predicts Class 1 (ASD). Otherwise, it predicts Class 0 (No ASD).



SVC (Support Vector Classifier) tries to separate two classes by drawing the **best possible boundary** between them.



In your project, the two classes are:

- 0 = No ASD
- 1 = ASD

Why RBF Kernel?

Your dataset likely cannot be separated with one straight line.

Because ASD prediction depends on combinations like:

- medium score + jaundice yes
- high score + no family history
- low score + many positive A1-A10 answers

So SVC uses **RBF kernel**.

RBF allows the model to draw a **curved boundary** instead of straight line.

That helps capture complex patterns.

1.ANN

Input Layer (features)



Dense 64 + ReLU

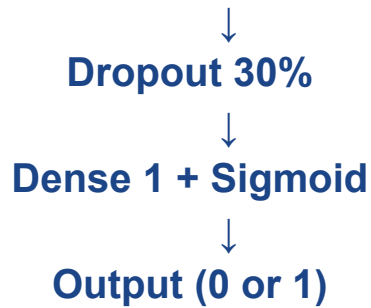


Dropout 30% -

(Randomly turns OFF 30% neurons during training.
Why? To prevent **overfitting**.)



Dense 32 + ReLU



#CODE -

```
model_ann = keras.Sequential([
```

Creates a neural network model using **Sequential API**.

Sequential means:

→ Layers stacked one after another:

Input → Hidden Layer 1 → Hidden Layer 2 → Output

Layer 1

```
layers.Dense(64, activation='relu', input_shape=(X_train_nn.shape[1],)),
```

Dense Layer Meaning

Dense = Fully connected layer

Every neuron connects to every neuron of previous layer.

64

Means:

→ 64 neurons in first hidden layer.

Each neuron learns different patterns.

Example -

1. If A1=1 and A3=1 and family autism=yes

→ higher ASD chance

2. Low result score + all zeros in answers

→ No ASD

◆ activation='relu'

Uses ReLU activation:

$$f(x) = \max(0, x)$$

Meaning:

- If input < 0 → output = 0
- If input > 0 → output = same value

Used because:

- ✓ Fast training
- ✓ Solves vanishing gradient problem

◆ Layer 2

`layers.Dropout(0.3),`

✓ Meaning:

Randomly turns OFF 30% neurons during training.

Why?

To prevent **overfitting**.

Example:

If 10 neurons:

→ 3 randomly disabled each training step.

Model learns more robustly.

Layer 3

`layers.Dense(32, activation='relu'),`

Second hidden layer:

- 32 neurons
- ReLU activation

Why fewer neurons?

Neural networks often reduce layer size gradually:

64 → 32 → 1 This compresses learned information.

Layer 4

`layers.Dropout(0.3),`

Again applies dropout.

30% neurons disabled randomly.

Output Layer

`layers.Dense(1, activation='sigmoid')`

Meaning:

Final layer has **1 neuron**

Used for binary classification:

- 0 = No
- 1 = Yes

◆ Sigmoid Activation

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Output between:

0 to 1

Example:

- 0.91 → class 1
- 0.12 → class 0

optimizer='adam'

Adam updates weights efficiently using gradients.

Very popular optimizer.

Think:

➡ Smart learning method.

ROC AUC Score

```
roc_auc_ann = metrics.roc_auc_score(Y_val_nn, Y_pred_ann_binary)
```

Measures classification quality.

Range:

- 0.5 = random guessing
- 1.0 = perfect model

loss='binary_crossentropy'

Used for binary classification.

Formula:

$$L = -(y \log(p) + (1-y) \log(1-p))$$

Where:

- y = actual label
- p = predicted probability

Lower loss = better model

epochs=50

Whole dataset used 50 times.

1 epoch = one full pass over data.

batch_size=32

Instead of using all data at once:

Uses 32 rows at a time.

Example:

1000 rows $\rightarrow$ 32 + 32 + 32...

This improves speed.

validation_split=0.2

20% training data used for validation.

If 1000 rows:

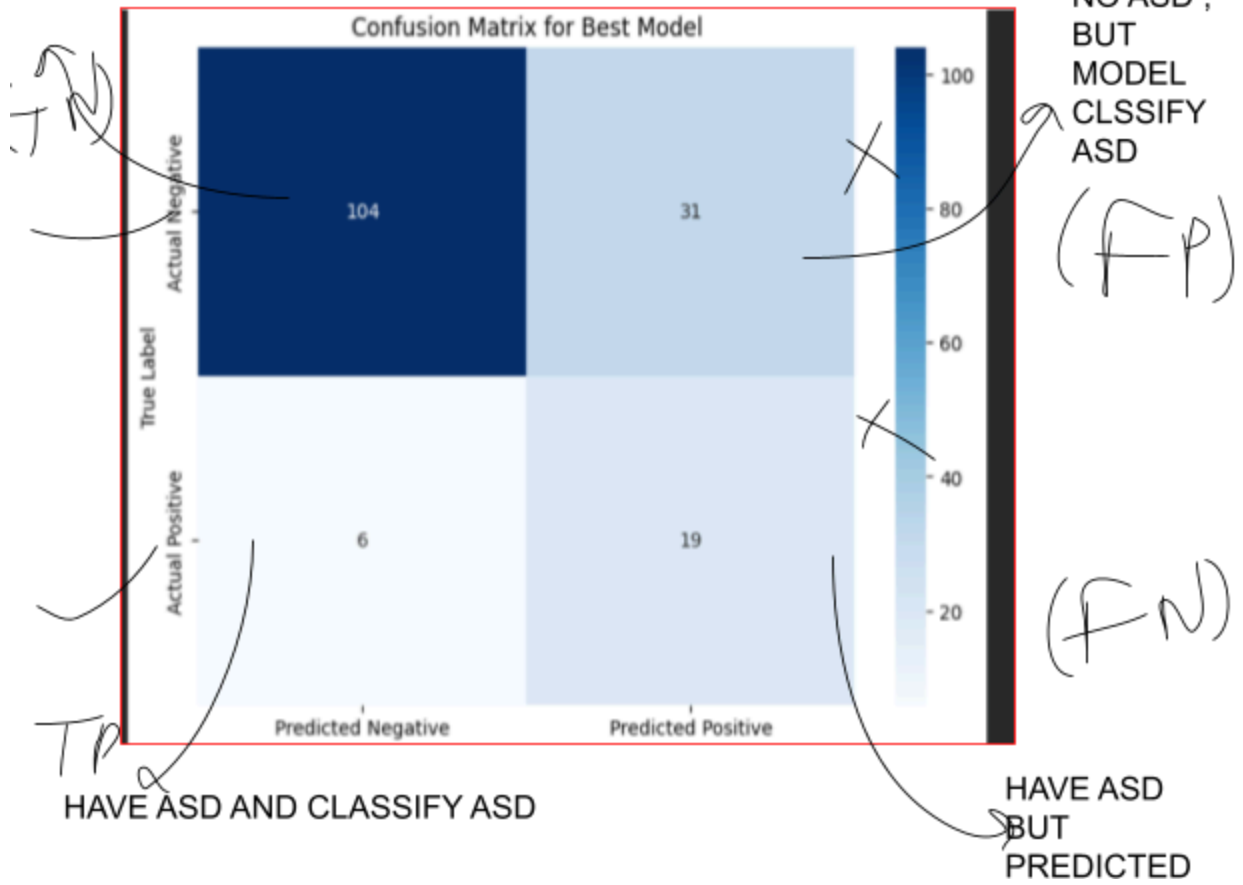
- 800 train
- 200 validate

verbose=0

No training progress shown.

Confusion Matrix

NO ASD AND MODEL CORRECTLY CLASSIFY



LSTM

Input (1 timestep, features)



LSTM 64



Dropout



Dense 32



Dropout



Sigmoid Output

```
X_train_lstm = X.values.reshape(X.shape[0], 1, X.shape[1])
Y_train_lstm = Y.values
X_val_lstm = X_val.values.reshape(X_val.shape[0], 1, X_val.shape[1])
Y_val_lstm = Y_val.values

# Define the LSTM model
```

```

model_lstm = keras.Se

X_train_lstm =
X.values.reshape(X.shape[0], 1,
X.shape[1])
Y_train_lstm = Y.values
X_val_lstm =
X_val.values.reshape(X_val.shape[0], 1,
X_val.shape[1])
Y_val_lstm = Y_val.values

# Define the LSTM model
model_lstm = keras.Sequential([
    layers.LSTM(64, activation='relu',
input_shape=(X_train_lstm.shape[1],
X_train_lstm.shape[2])),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])

model_lstm.compile(optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy'])

quential([
    layers.LSTM(64, activation='relu', input_shape=(X_train_lstm.shape[1],
X_train_lstm.shape[2])),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])

model_lstm.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

history_lstm = model_lstm.fit(X_train_lstm, Y_train_lstm, epochs=50,
batch_size=32, validation_split=0.2, verbose=0)

print("LSTM model trained successfully!")

```

RESHAPE TO 3D
(samples,
timesteps,
features)

One
person's
attributes
at one
moment

1, no of feature (1,21)

Read input
remember past
at each time
stamp

```
# Evaluate the LSTM model
loss_lstm, accuracy_lstm = model_lstm.evaluate(X_val_lstm, Y_val_lstm,
verbose=0)
print(f'Validation Accuracy (LSTM): {accuracy_lstm:.4f}')

Y_pred_lstm = model_lstm.predict(X_val_lstm)
Y_pred_lstm_binary = (Y_pred_lstm > 0.5).astype(int)

roc_auc_lstm = metrics.roc_auc_score(Y_val_lstm, Y_pred_lstm_binary)
print(f'Validation ROC AUC (LSTM): {roc_auc_lstm:.4f}')
```